SENG 300 – Assignment 1 Report

Monday, October 19, 2026

William Vandergraaf – 30222999
Kahan Raval – 30245804
Elyas Hossaini – 30206499
Javad Mammadaiyev – 30268088
Yogita Bamrah – 30265974

GoCycle Requirements Classification

Functional Requirements

These are requirements that cover registration, trips, pricing, payments, maintenance, and operator control.

Item	Requirement
Rider Accounts	Casual, monthly, student riders
Rider Billing	Charge monthly members a monthly membership payment
Starting Trips	Allow riders to select and rent available vehicles; start trip from phone app or screen and card reader; prevent rental of vehicle with less than 20% charge
Pricing	Calculate trip price using per-minute charge; apply a lower per-minute charge to monthly members and to students; apply long-trip surcharge when trip time is over 2 hours; charge an unlock fee, plus a per-minute fee
Returning Vehicles	Allow riders to return vehicle to any station with free docking point; verify vehicle is locked; if station is full, redirect to nearest non-full station; keep trip active and charging until vehicle is locked; release deposit when trip ends
Payments	Payments via credit cards, city transit cards, phone wallets; only allow permitted payment methods depending on zone; hold a refundable deposit when a trip starts
Receipts	Provide a receipt through app or by email when trip is finished and paid
Fault Reporting	Allow riders to report vehicle problems; receive automatic low-battery and fault reports from electric vehicles; report problems to control room operators
Fault Handling	Remove broken and low-battery devices from rental pool; allow control room operators to manipulate fault reports
Operator Control	Display the state of every station and vehicle; support operators in relocating vehicles between stations; allow operators to take vehicles in and out of service
Physical Screens	Support more than one language; have audio outputs
Zone Config	Display each zone's colours and logo, pulled and changed via configuration

Quality Requirements

These are requirements that cover usability, accessibility, and pricing.

Item	Requirement
Usability	Rental and return workflows should be straightforward and easy for all users, regardless of whether they are using the app or the screens
Accessibility	Interface should accommodate different language and accessibility needs, including people without the app
Extensibility	Pricing should be easy to extend when new categories are introduced

Constraints

These are the boundaries of the application and the GoCycle operating environment.

Item	Requirement
Docking Capacity	Each docking point can hold only one vehicle
Vehicle Scope	Three types: pedal bikes, electric bikes, electric scooters
Hardware	Stations with small screen and card reader, and app access
Payment Scope	Three options: credit cards, city transit cards, mobile wallets
Zone Rules	Separate zones may restrict payment methods, will have custom logos/colours
Development Constraints	Java, SOLID principles, at least two design patterns, at least eight classes, implementation and design in shared git repo

GoCycle Assumptions

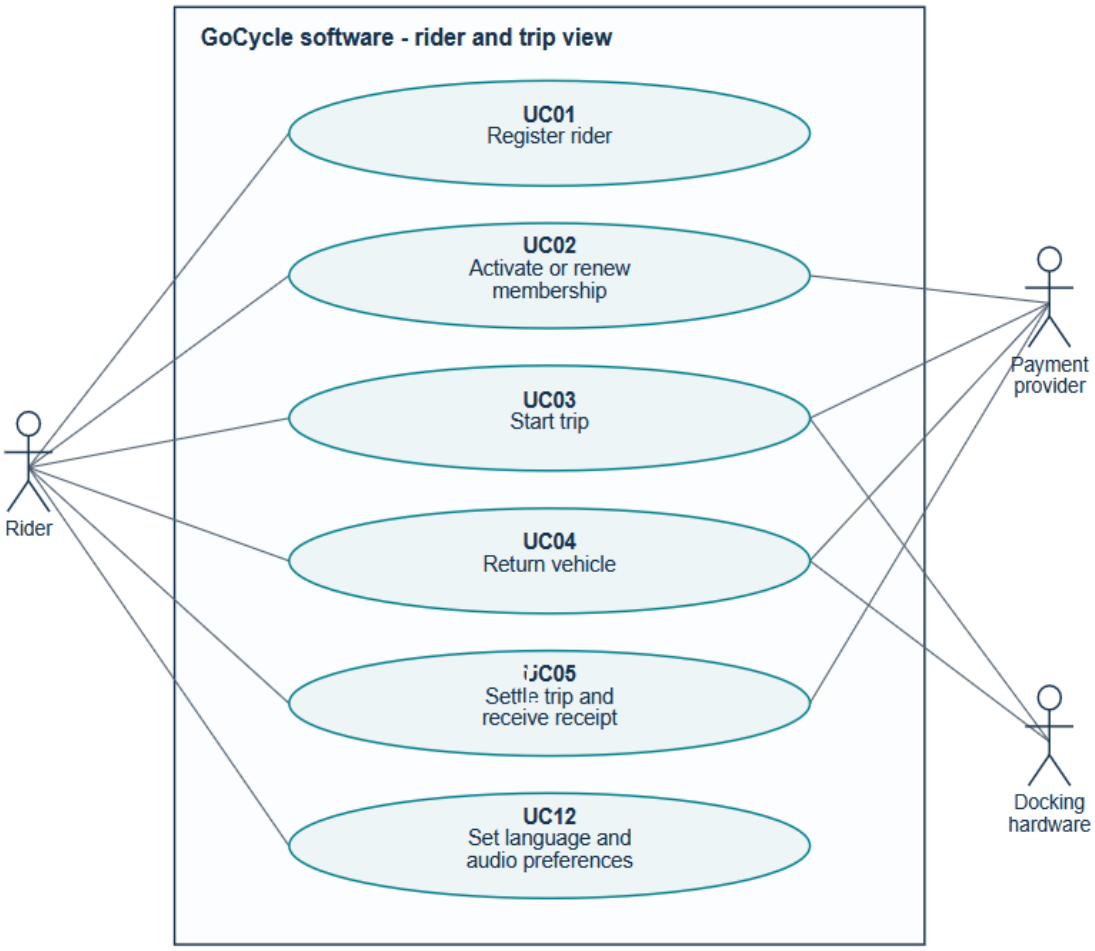
There are also a series of business constraints that need to be decided before beginning implementation.

Item	Requirement
Unlock fee?	$5
Casual, member, and student rates?	$0.50/minute, $0.20/minute, $0.20/minute
Pedal bike, electric bike, scooter prices?	Same for all
Refundable deposit amount?	$50
Time unit?	Per minute, round up
Minimum trip charge?	No
Long-trip surcharge rate?	$15
Who qualifies as a student?	Anyone with an `@ucalgary.ca` email address
Age restriction on students?	No
Can students also be monthly members?	No
Does being a student affect all rates?	No, only the per minute rate
Monthly membership price?	$15/month
Membership duration?	30 days from time of registration
Does every rider have to have an account?	Yes
What information is required to register?	Email/username, password, account type (casual, member, student), payment method
How do riders identify themselves at a station?	Login with email/username and password
Can one account do multiple trips?	No
When does billable time start?	As soon as vehicle is unlocked
How long does rider have to remove unlocked vehicle?	2 minutes
What happens if rider does not remove vehicle?	After 2 minutes, trip stopped, no charge
Can trips be paused?	No
Maximum trip duration?	12 hours, if not returned, vehicle is considered lost
What happens if vehicle is never returned?	Vehicle considered lost, operators notified, deposit is charged
Can an operator manually close a trip?	Yes
How are lost or stolen devices handled?	Vehicle is marked lost, cannot be rented, deposit charged to last user
What happens if a device runs out of battery during a trip?	Direct user to nearest station to receive charge

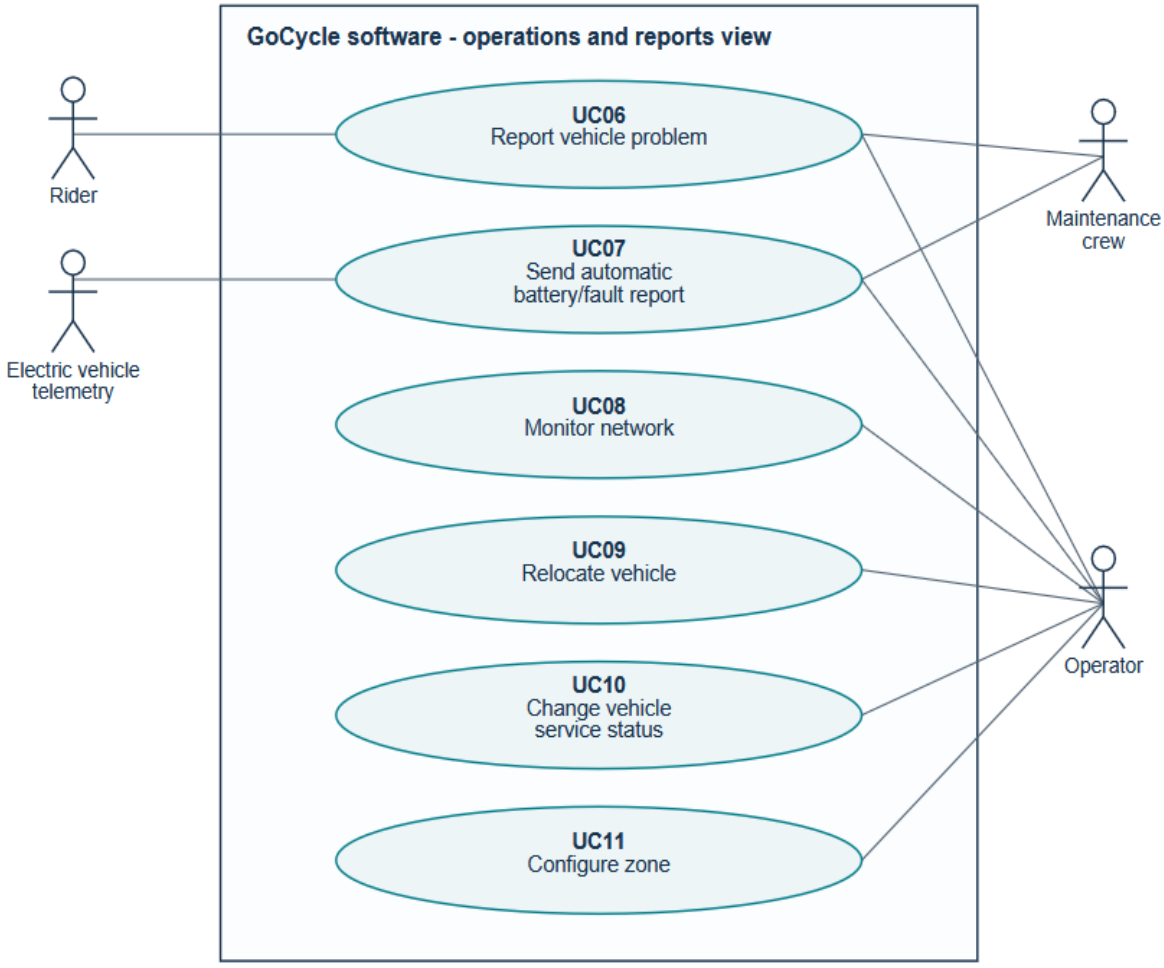
How many stations?	5 (3 in one zone, 2 in another)
How many docking points per station?	5
Can every docking point accept every vehicle?	Yes
Does a fault report immediately take the vehicle out of service?	Yes, after current ride is completed
Can a rider use a vehicle reported as broken?	No
Can operators override battery restrictions or faults?	Yes
What happens to duplicate fault reports?	Discarded
What languages?	English, French
What is the default language?	English
How is audio enabled?	Pressing a button on the screen
Where do receipts go?	Email and mobile app
What information appears on receipt?	Unlock fee, duration, rate, surcharge, and deposit release

GoCycle Use Case Model

Rider and Trip View Use Case Diagram



Operations and Reporting View Use Case Diagram



Use Case 1: Start Trip

User: Rider
Trigger: A rider selects a vehicle and requests a rental.
Preconditions: Rider is logged in, the station and selected vehicle type.

1. Rider chooses a vehicle through the app or station screen
2. GoCycle checks that the vehicle is available and not out of service
    a. For an electric vehicle, it checks the battery reading
    b. If battery is greater than or equal to 20%, vehicle is reserved
    c. If the vehicle is already rented or out of service, request is rejected and another selection must be made
    d. If the battery is less than 20%, reject the rental and another selection must be made
3. GoCycle determines the pricing and presents it to the rider
4. The Rider selects a payment method, GoCycle checks that this payment method is valid for the zone
    a. If not valid, payment is rejected and user must select a different method
5. GoCycle requests a refundable deposit, and checks that this payment method is valid for the zone
    a. If not valid, payment is rejected and user must select a different method
6. GoCycle requests unlocking from the dock and receives confirmation
    a. If unlocking fails, do not start billing, reject request and another selection must be made
7. GoCycle records the trip as active and starts billable time, and marks the vehicle as rented

Success: Rider and vehicle are linked for an active trip, deposit is recorded, billable time starts, vehicle is unavailable.
Failure: Rejected requests/unlocking errors do not start billable time.

Use Case 2: Return Vehicle

User: Rider
Trigger: A rider returns a rented vehicle to end a trip.
Preconditions: The vehicle is on an active trip.

1. GoCycle checks for a usable free docking point at the destination station
    a. If the station has no free docking point, GoCycle identifies the nearest station with one and redirects the rider there – the trip remains active and charges continue while redirecting
    b. This action can continue/loop until there is a free docking station
2. The rider places the vehicle in a free docking point and locks it
3. GoCycle receives confirmation that the vehicle is locked
    a. If confirmation fails, do not stop charges or end trip, and tell user to retry locking the vehicle
4. After confirmation, GoCycle records the trip end time and stops billable time
5. GoCycle releases the deposit on hold
6. GoCycle records the vehicle at the docking point
    a. For an electric vehicle, it checks the battery reading
    b. If the battery level is greater than or equal to 20%, the vehicle is available for rental, if the battery level is less than 20% the vehicle is not available
7. GoCycle calculates the fare and confirms it has been successfully collected before issuing the receipt to the app and the user's email

Success: Trip is ended, billable time is stopped, station occupancy is updated, and the vehicle is ready for rental again.
Failure: Trip remains active until user successfully locks the vehicle.

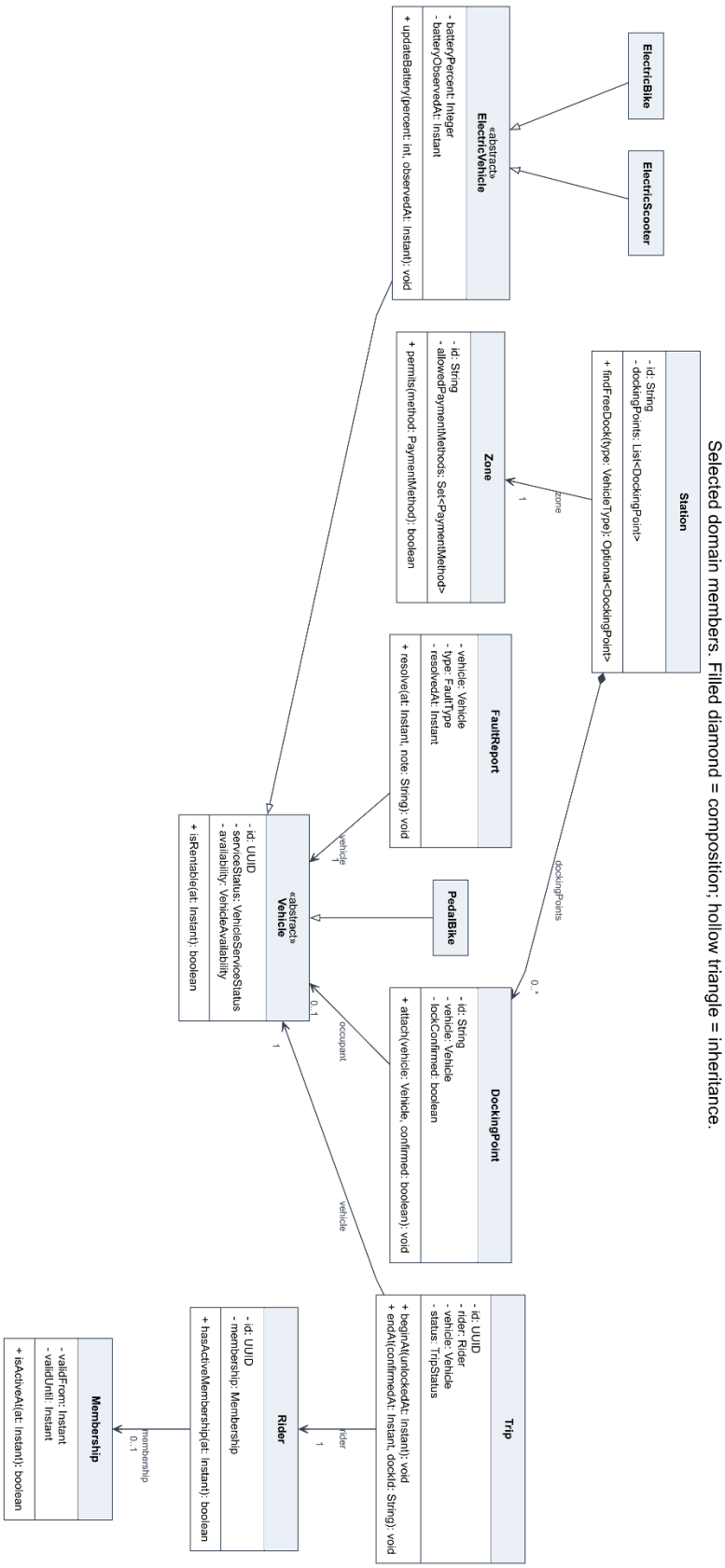
Use Case 3: Report Vehicle Problem

User: Rider
Trigger: A rider submits a report of a broken or faulty vehicle.
Preconditions: The vehicle can be identified.

1. The rider identifies the vehicle and selects the problem from a list of possible problems
2. GoCycle records the report
    a. If the report is a duplicate, it is discarded
3. GoCycle changes the status of the vehicle to out of service and prevents further rentals
    a. If the vehicle is currently rented, prevent rentals after it is returned to a station
4. GoCycle notifies Control Room and maintenance staff
5. GoCycle acknowledges that the report was received

Success: The report is recorded, the broken vehicle cannot be rented again, and operators can see the issue.
Failure: The user is notified that their report could not be received, and they are encouraged to try again.

UML Class Diagram

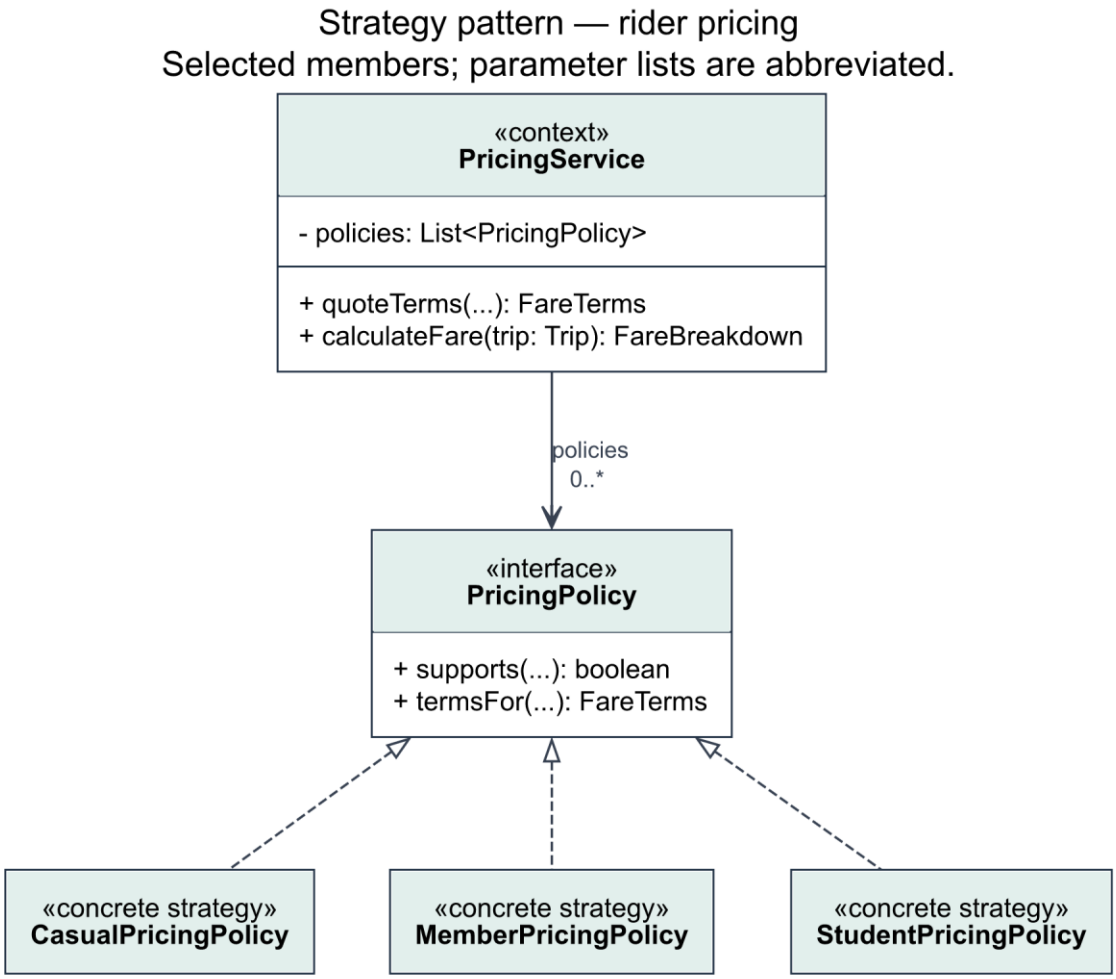


Design Rationale

The SOLID Design principles are present throughout all the classes in the project. We used the Strategy and Observer patterns to solve problems. A problem we have is that different rider categories have different prices, and the client expects additional categories to be added later, which is a good fit for the Strategy pattern. The Strategy pattern has the benefits of keeping pricing rules separate and makes extending rider categories in the future easier, while having the trade-offs of introducing more classes, and stricter testing requirements. Classes that fall into this pattern are as follows:

- `PricingService`
- `PricingPolicy`
- `CasualPricingPolicy`
- `MemberPricingPolicy`
- `StudentPricingPolicy`

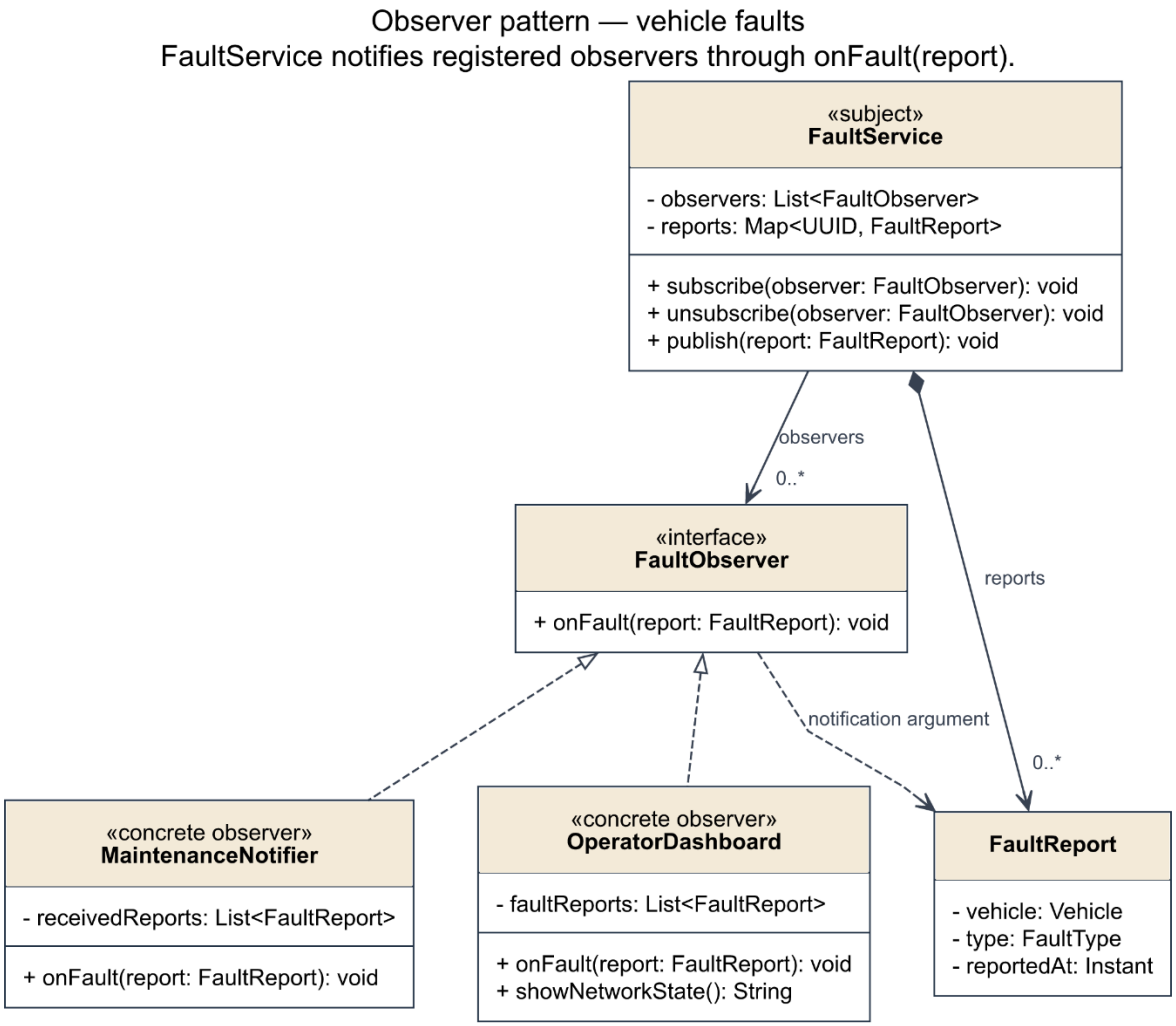
Here is the UML diagram for the Strategy pattern:



When a vehicle is reported as broken, it is important to notify all affected participants, in this case, that is the maintenance teams and operators. Additional recipients should be possible later, without rewriting how faults are handled. The Observer design pattern is a good fit for this situation, and it has the following benefits: separates fault reporting from maintenance and dashboard notifications, and new recipients can be added easily. The Observer design pattern also has some trade-offs: notification failures, retries, and duplicate delivery will require additional handling. We used the Observer design pattern in the following classes:

- `FaultService`
- `FaultObserver`
- `MaintenanceNotifier`
- `OperatorDashboard`
- `FaultReport`

Here is the UML diagram for the Observer pattern:

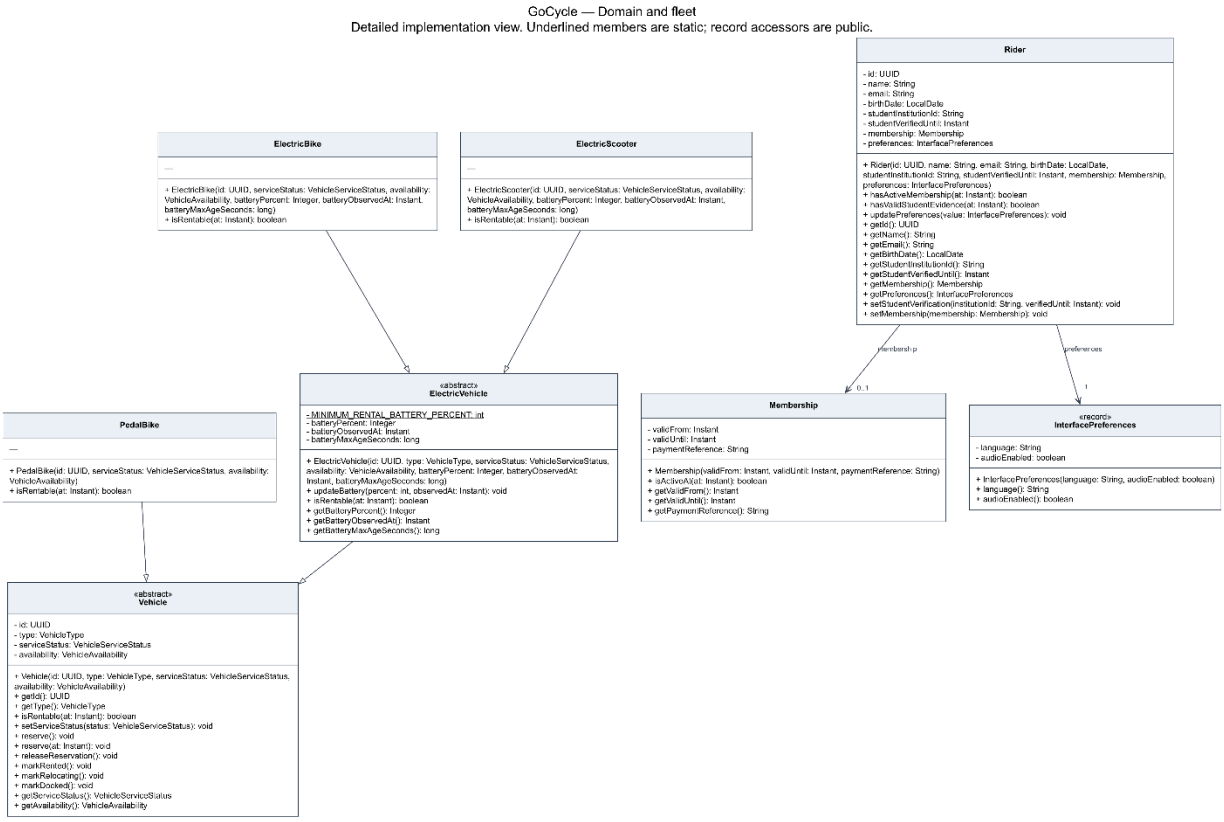


Here is how we implemented each letter of the SOLID principles:

S	Pricing, payments, trip coordination, fault processing, and presentation have separate owners
O	New pricing policies and fault observers can be added without modifying the services that use them; pricing, categories, and zones are configured during application setup, rather than hardcoded
L	All vehicle subclasses stem from the common `Vehicle` class; battery-based devices only are further separated into the `ElectricVehicle` subclass
I	Payment, docking, receipt delivery, and fault notification have separate, focused interfaces
D	Services receive interfaces through constructors; `TripService` depends on `DockingGateway`, allowing simulated hardware to be replaced with real

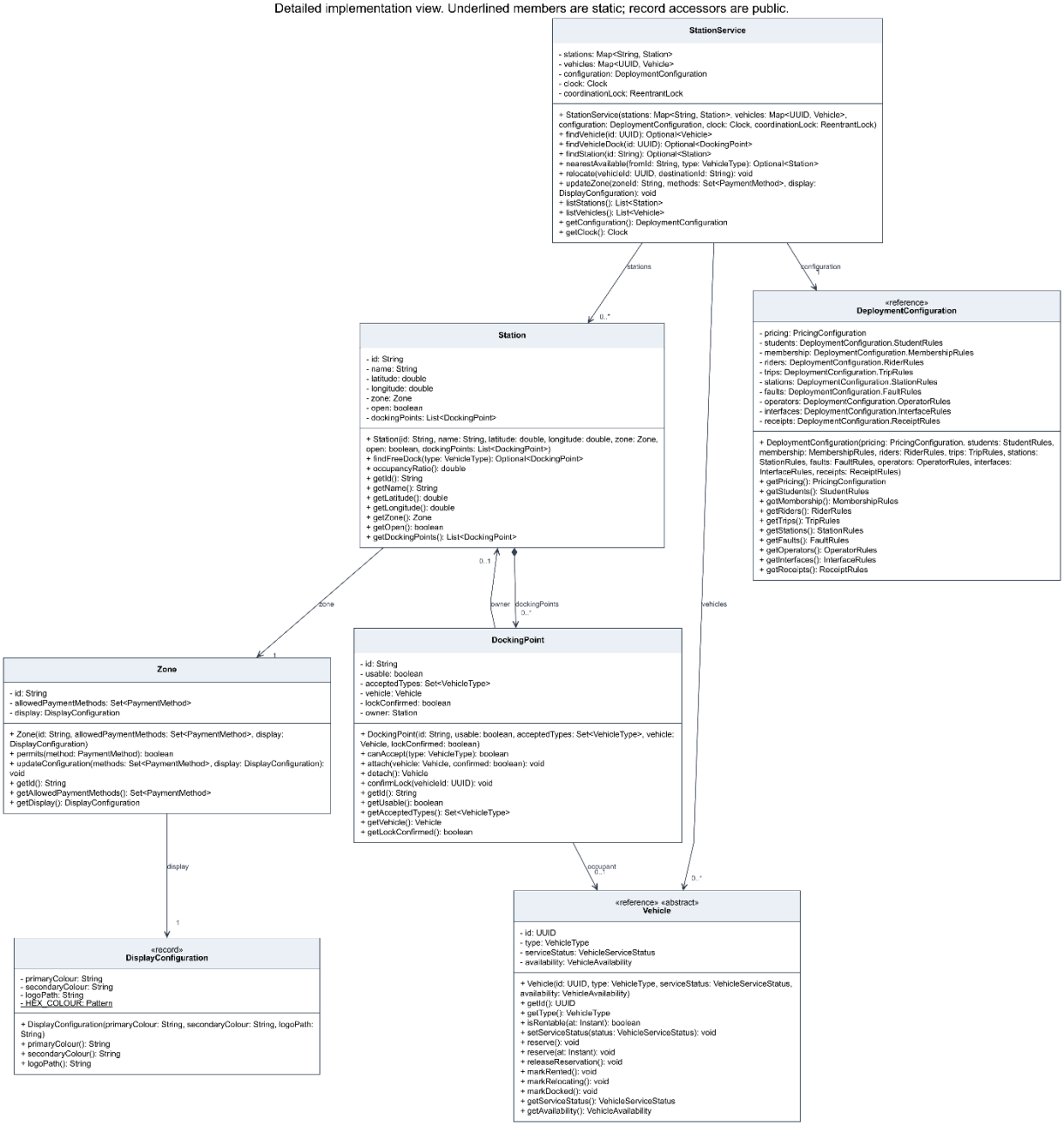
Our SOLID-based design separates trip management, pricing, payments, and fault reporting so that way changes in one area of the codebase have less effect on others. The Strategy pricing pattern gives each rider its own pricing policy, so new categories can be easily added, without changing how trips work. The Observer pattern lets fault reporting notify maintenance staff and operators without depending on how those recipients handle notifications. New recipients can be easily added, without changing the overall process. Using small, focused interfaces for payments, docking, and receipt delivery also let us test those services in a more easy manner, and allow us to use simulations in testing that can be replaced with real integrations in a real-world deployment.

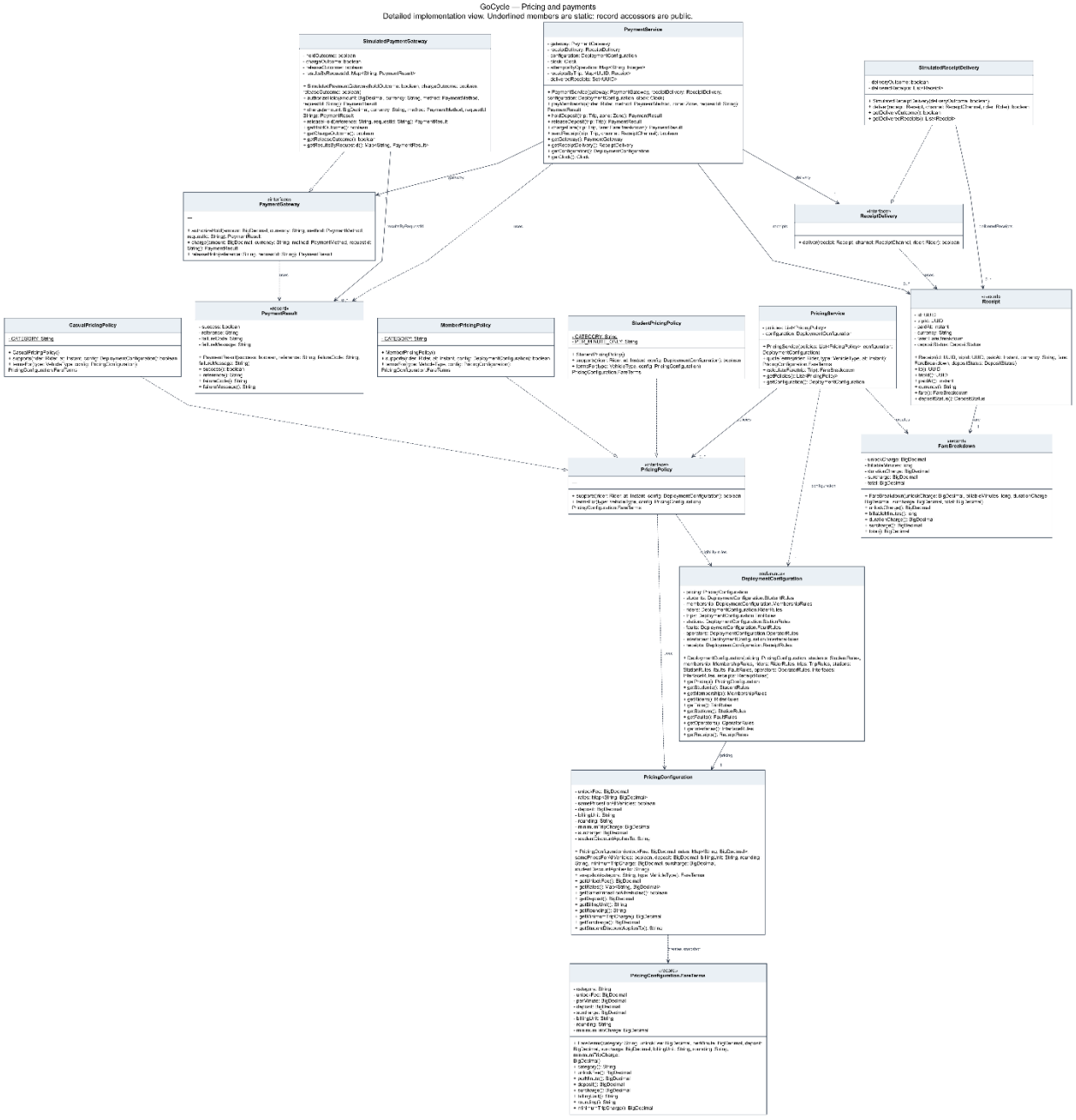
Appendix A: Additional UML Diagrams

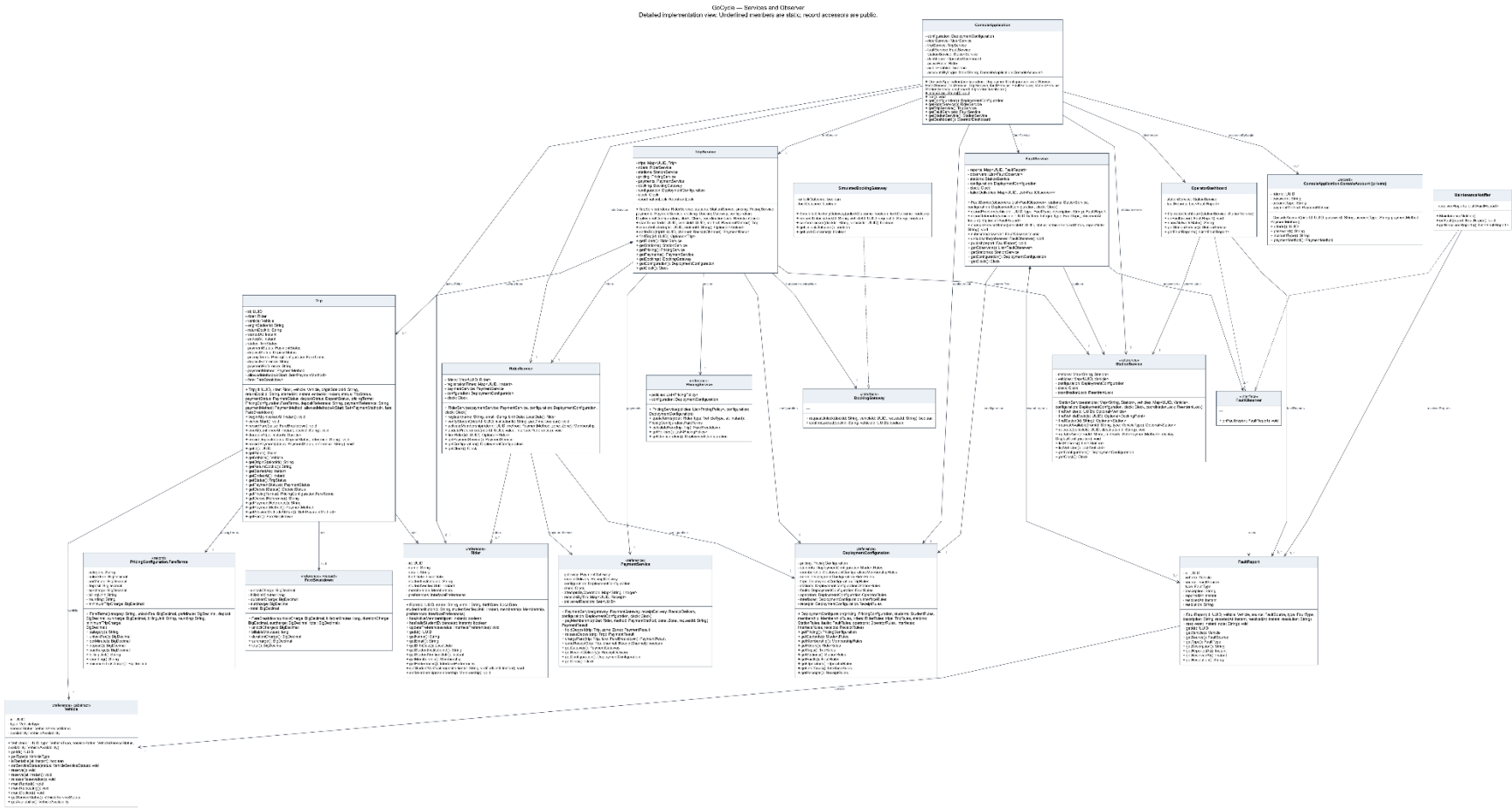


GoCycle — Stations

Detailed implementation view. Underlined members are static; record accessors are public.




GoCycle — Pricing and payments
Detailed implementation view. Underlined members are static; record accessors are public.
SimulatedPaymentGateway
- failOutcome: boolean
- chargeOutcome: boolean
- refundOutcome: boolean
- nextAuthResult(): MapString, PaymentResult>
+ SimulatedPaymentGateway(boolean, boolean, boolean)
+ authorize(idempotencyKey: String, amount: BigDecimal, currency: String, method: PaymentMethod, requestId: String): PaymentResult
+ charge(idempotencyKey: String, amount: BigDecimal, currency: String, method: PaymentMethod, requestId: String): PaymentResult
+ refund(providerTxnId: String, amount: BigDecimal, requestId: String): PaymentResult
+ void(providerTxnId: String, requestId: String): PaymentResult
+ setNextResult(requestId: String, result: PaymentResult)
PaymentService
- gateway: PaymentGateway
- receiptDelivery: ReceiptDelivery
- configuration: DeploymentConfiguration
- clock: Clock
- idempotencyCache: MapString, Integer>
- nextAuthTxn: Map<UUID, PaymentResult>
- deliveredReceipts: SetUUID>
+ PaymentService(PaymentGateway, ReceiptDelivery, DeploymentConfiguration, Clock)
+ payAndReserve(rider: Rider, method: PaymentMethod, ride: Ride, amount: BigDecimal): PaymentResult
+ holdDeposit(trip: Trip, amount: BigDecimal): PaymentResult
+ releaseDeposit(trip: Trip): PaymentResult
+ chargeLate(trip: Trip, lateFee: BigDecimal): PaymentResult
+ resolveDispute(trip: Trip, customerIsRight: boolean): PaymentResult
+ setGateway(gateway: PaymentGateway)
+ getReceiptDelivery(): ReceiptDelivery
+ getDeployment(): DeploymentConfiguration
+ getClock(): Clock
SimulatedReceiptDelivery
- deliveryOutcome: boolean
- delivered: Map<UUID, List<Receipt>>
+ SimulatedReceiptDelivery(boolean)
+ deliver(receipt: Receipt, channel: ReceiptChannel, rider: Rider): boolean
+ wasDelivered(): boolean
+ getDeliveredReceipts(): List<Receipt>
«interface»
PaymentGateway
+ authorize(idempotencyKey: String, amount: BigDecimal, currency: String, method: PaymentMethod, requestId: String): PaymentResult
+ charge(idempotencyKey: String, amount: BigDecimal, currency: String, method: PaymentMethod, requestId: String): PaymentResult
+ refund(idempotencyKey: String, amount: BigDecimal, requestId: String): PaymentResult
«interface»
ReceiptDelivery
+ deliver(receipt: Receipt, channel: ReceiptChannel, rider: Rider): boolean
«record»
PaymentResult
- success: boolean
- reference: String
- failureCode: String
- failureMessage: String
+ PaymentResult(boolean, boolean, reference: String, failureCode: String, failureMessage: String)
+ succeeded(boolean, reference: String): PaymentResult
+ declined(failureCode: String, failureMessage: String): PaymentResult
«record»
Receipt
- id: UUID
- rideId: UUID
- riderId: UUID
- currency: String
- amount breakdown
- issued: Instant, channel: String
+ Receipt(id: UUID, rideId: UUID, riderId: UUID, issued: Instant, currency: String, line: FareBreakdown, depositStatus: DepositStatus)
+ id(): UUID
+ rideId(): UUID
+ riderId(): UUID
+ issued(): Instant
+ currency(): String
+ line(): FareBreakdown
+ depositStatus(): DepositStatus
«record»
FareBreakdown
- baseCharge: BigDecimal
- distanceCharge: BigDecimal
- timeCharge: BigDecimal
- surcharge: BigDecimal
total: BigDecimal
+ FareBreakdown(baseCharge: BigDecimal, distanceFeeAtRate: long, durationCharge: BigDecimal, surcharge: BigDecimal, total: long, reserved)
+ subtotal(): BigDecimal
+ toMultiline(): String
+ distanceCharge(): BigDecimal
+ total(): BigDecimal
«interface»
PricingPolicy
+ suspendsAt(ride: Ride, at: Instant, config: DeploymentConfiguration): boolean
+ name(): String, vehicleType(): String, config(): PricingConfiguration
«interface»
DeploymentConfiguration
- pricing: PricingConfiguration
- clock: SystemClockOperationStateRules
- fraudRules: DispatchOperationVerificationRules
- fees: DeploymentConfigurationFeeRules
- hours: DeploymentConfigurationHoursRules
- stations: DeploymentConfigurationStationRules
- tools: DispatchOperationFaultRules
operations: DeploymentConfigurationOperationRules
- customer: DeploymentConfigurationCustomerRules
- vehicles: DeploymentConfigurationVehicleRules
+ DeploymentConfiguration(pricing: PricingConfiguration, stations: StationRules, fraudRules: MotionDutyRules, fees: FeeRules, hours: TripRules, clock: StationRules, faults: FaultRules, operations: OperationRules, customer: CustomerRules, vehicles: VehicleRules)
+ withPricing(pricing: PricingConfiguration): DeploymentConfiguration
+ pricing(): PricingConfiguration
+ getStations(): StationRules
+ getMaintenance(): MaintenanceRules
+ getHours(): TripRules
+ getClock(): Clock
+ getFaults(): FaultRules
+ getOperations(): OperationRules
+ getCustomer(): CustomerRules
+ getVehicles(): VehicleRules
+ getFees(): FeeRules
PricingConfiguration
- unlockFee: BigDecimal
- perMinuteRate: BigDecimal
- peakMultiplier: BigDecimal
- discountPerMile: BigDecimal
- minimumTripCharge: BigDecimal
- surcharge: BigDecimal
- lowBatteryThreshold: Integer
+ PricingConfiguration(unlockFee: BigDecimal, perMinuteRate: BigDecimal, peakMultiplier: BigDecimal, discountPerMile: BigDecimal, minimumTripCharge: BigDecimal, surcharge: BigDecimal, lowBatteryThreshold: Integer)
+ withUnlockFee(): BigDecimal
+ getRates(): MapString, BigDecimal>
+ getSurcharge(): BigDecimal
+ getDeposit(): BigDecimal
+ withRate(): String
+ getDiscounting(): String
+ getMinimumTripCharge(): BigDecimal
+ getSurcharge(): BigDecimal
+ withDiscountAppliedTo(): String
«record»
PricingConfiguration.FareTerms
- category: String
- unlockFee: BigDecimal
- perMinute: BigDecimal
- deposit: BigDecimal
- surcharge: BigDecimal
- billingUnit: String
- roaming: String
- minimumCharge: BigDecimal
+ FareTerms(category: String, unlock: int, big terms, rateMinute: BigDecimal, deposit: BigDecimal, surcharge: BigDecimal, billingUnit: String, currency: String, minimumTripCharge: BigDecimal)
BigDecimal()
+ category(): String
+ unlockFee(): BigDecimal
+ perMinute(): BigDecimal
+ deposit(): BigDecimal
+ surcharge(): BigDecimal
+ billingUnit(): String
+ roaming(): String
+ minimumTripCharge(): BigDecimal
CasualPricingPolicy
- CATEGORY: String
+ CasualPricingPolicy()
+ suspendsAt(ride: Ride, at: Instant, config: DeploymentConfiguration): boolean
+ name(): String, vehicleType(): String, config(): PricingConfiguration
PricingConfiguration.FareTerms
MemberPricingPolicy
- CATEGORY: String
+ MemberPricingPolicy()
+ suspendsAt(ride: Ride, at: Instant, config: DeploymentConfiguration): boolean
+ name(): String, vehicleType(): String, config(): PricingConfiguration
PricingConfiguration.FareTerms
StudentPricingPolicy
- CATEGORY: String
- DISCOUNT: double
+ StudentPricingPolicy()
+ suspendsAt(ride: Ride, at: Instant, config: DeploymentConfiguration): boolean
+ name(): String, vehicleType(): String, config(): PricingConfiguration
PricingConfiguration.FareTerms
PricingService
- unlockFeeOverride: Integer
- configuration: DeploymentConfiguration
+ PricingService(policies: List<PricingPolicy>, configuration: DeploymentConfiguration)
+ quote(reservation: Reservation, vehicleType: VehicleType, at: Instant): PricingConfiguration.FareTerms
+ calculateFare(ride: Trip): FareBreakdown
+ getPolicies(): List<PricingPolicy>
+ getConfiguration(): DeploymentConfiguration
+ withUnlockFeeOverride(fee: Integer): PricingService
+ withConfiguration(configuration: DeploymentConfiguration): PricingService
+ withPolicies(policies: List<PricingPolicy>): PricingService
+ getUnlockFeeOverride(): Integer
+ getConfiguration(): DeploymentConfiguration
+ getPolicies(): List<PricingPolicy>
+ toString(): String
+ equals(Object): boolean
+ hashCode(): int
+ clone(): PricingService
+ finalize(): void
+ getClass(): Class<?>
+ notify(): void
+ notifyAll(): void
+ wait(long, int): void
+ wait(long): void
+ wait(): void


UseCycle — Services and Observer
Detailed implementation view. Underlined members are static; record accessors are public.
ConsoleApplication
- configLoader: DeploymentConfigLoader
- stripGateway: StripGateway
- fraudScreen: FraudScreen
- bookingGateway: BookingGateway
- tripService: TripService
- invoiceStore: Map<UUID, Invoice>
- observers: List<TripObserver>
+ ConsoleApplication(configLoader: DeploymentConfigLoader, stripGateway: StripGateway, fraudScreen: FraudScreen, bookingGateway: BookingGateway, tripService: TripService, invoiceStore: Map<UUID, Invoice>, observers: List<TripObserver>)
+ configLoader(): DeploymentConfigLoader
+ stripGateway(): StripGateway
+ fraudScreen(): FraudScreen
+ bookingGateway(): BookingGateway
+ tripService(): TripService
+ invoiceStore(): Map<UUID, Invoice>
+ observers(): List<TripObserver>
+ equals(Object): boolean
+ hashCode(): int
+ toString(): String
TripService
- trips: Map<UUID, Trip>
- clock: TripClock
- pricing: PricingService
- payments: PaymentService
- fraud: FraudScreen
- configLoader: DeploymentConfigLoader
- audit: AuditLog
- nextId: AtomicLong
+ TripService(pricing: PricingService, payments: PaymentService, fraud: FraudScreen, configLoader: DeploymentConfigLoader, clock: TripClock, audit: AuditLog)
+ TripService(pricing: PricingService, payments: PaymentService, fraud: FraudScreen, configLoader: DeploymentConfigLoader, clock: TripClock)
+ TripService(pricing: PricingService, payments: PaymentService, fraud: FraudScreen, configLoader: DeploymentConfigLoader)
+ create(rider: String, pickup: String, dropoff: String, distanceKm: double, surge: double): Trip
+ get(id: UUID): Trip
+ find(id: UUID): Optional<Trip>
+ all(): List<Trip>
+ byStatus(status: TripStatus): List<Trip>
+ assign(id: UUID, driver: String): Trip
+ start(id: UUID): Trip
+ complete(id: UUID): Trip
+ cancel(id: UUID, reason: String): Trip
+ refund(id: UUID): Trip
+ quote(pickup: String, dropoff: String, distanceKm: double, surge: double): Money
+ pricing(): PricingService
+ payments(): PaymentService
+ fraud(): FraudScreen
+ configLoader(): DeploymentConfigLoader
+ audit(): AuditLog
+ clock(): TripClock
SmartphoneBookingGateway
- serviceBaseUrl: String
- client: Object
+ SmartphoneBookingGateway(serviceBaseUrl: String)
+ requestRide(riderId: String, pickup: String, dropoff: String): String
+ cancelRide(rideId: String): void
+ rideStatus(rideId: String): String
FraudScreen
- reports: Map<UUID, FraudReport>
- clock: TripClock
- policy: DeploymentConfig
- history: Map<UUID, List<Instant>>
+ FraudScreen(clock: TripClock, policy: DeploymentConfig)
+ FraudScreen(clock: TripClock)
+ FraudScreen(policy: DeploymentConfig)
+ FraudScreen()
+ fileReport(tripId: UUID, riderId: String, amount: Money, ipAddress: String): FraudReport
+ review(tripId: UUID, approve: boolean, analyst: String, notes: String): FraudReport
+ reportFor(tripId: UUID): Optional<FraudReport>
+ pendingReports(): List<FraudReport>
+ velocityCheck(riderId: String, now: Instant): boolean
+ policy(): DeploymentConfig
+ clock(): TripClock
OperatorDashboard
- stripGateway: StripGateway
- fraudScreen: FraudScreen
+ OperatorDashboard(stripGateway: StripGateway, fraudScreen: FraudScreen)
+ renderRevenue(): String
+ renderFraudQueue(): String
+ refundTrip(tripId: UUID): String
+ renderPayments(): String
ConsoleApplication.ConsoleAccount (private record)
- token: UUID
- name: String
- email: String
- createdAt: Instant
+ ConsoleAccount(token: UUID, name: String, email: String, createdAt: Instant)
+ token(): UUID
+ name(): String
+ email(): String
+ createdAt(): Instant
+ equals(Object): boolean
+ hashCode(): int
+ toString(): String
StartInsuranceOffer
- insuranceStripe: StripeGateway
+ StartInsuranceOffer(insuranceStripe: StripeGateway)
+ apply(trip: Trip, riderEmail: String): String
Trip
- id: UUID
- rider: Rider
- pickup: Address
- dropoff: Address
- distanceKm: double
- surge: double
- quote: Money
- status: TripStatus
- driver: String
- paymentRef: String
- createdAt: Instant
- updatedAt: Instant
+ Trip(id: UUID, rider: Rider, pickup: Address, dropoff: Address, distanceKm: double, surge: double, quote: Money, status: TripStatus, driver: String, paymentRef: String, createdAt: Instant, updatedAt: Instant)
+ Trip(id: UUID, rider: Rider, pickup: Address, dropoff: Address, distanceKm: double, surge: double, quote: Money)
+ id(): UUID
+ rider(): Rider
+ pickup(): Address
+ dropoff(): Address
+ distanceKm(): double
+ surge(): double
+ quote(): Money
+ status(): TripStatus
+ driver(): String
+ paymentRef(): String
+ createdAt(): Instant
+ updatedAt(): Instant
+ assign(driver: String, now: Instant): void
+ start(now: Instant): void
+ complete(now: Instant): void
+ cancel(now: Instant): void
+ refund(now: Instant): void
+ attachPayment(ref: String, now: Instant): void
+ isActive(): boolean
+ isTerminal(): boolean
+ equals(Object): boolean
+ hashCode(): int
+ toString(): String
RideReference
- trips: Map<UUID, Trip>
- clock: TripClock
- pricing: PricingService
- payments: PaymentService
- fraud: FraudScreen
- configLoader: DeploymentConfigLoader
- audit: AuditLog
+ RideReference(pricing: PricingService, payments: PaymentService, fraud: FraudScreen, configLoader: DeploymentConfigLoader, clock: TripClock, audit: AuditLog)
+ RideReference(pricing: PricingService, payments: PaymentService, fraud: FraudScreen, configLoader: DeploymentConfigLoader, clock: TripClock)
+ RideReference(pricing: PricingService, payments: PaymentService, fraud: FraudScreen, configLoader: DeploymentConfigLoader)
+ create(rider: String, pickup: String, dropoff: String, distanceKm: double, surge: double): Trip
+ get(id: UUID): Trip
+ find(id: UUID): Optional<Trip>
+ all(): List<Trip>
+ byStatus(status: TripStatus): List<Trip>
+ assign(id: UUID, driver: String): Trip
+ start(id: UUID): Trip
+ complete(id: UUID): Trip
+ cancel(id: UUID, reason: String): Trip
+ refund(id: UUID): Trip
+ quote(pickup: String, dropoff: String, distanceKm: double, surge: double): Money
+ pricing(): PricingService
+ payments(): PaymentService
+ fraud(): FraudScreen
+ configLoader(): DeploymentConfigLoader
+ clock(): TripClock
+ audit(): AuditLog
interface
PricingGateway
+ quote(pickup: String, dropoff: String, distanceKm: double, surge: double): Money
+ configLoader: DeploymentConfigLoader
interface
BookingGateway
+ requestRide(riderId: String, pickup: String, dropoff: String): String
+ cancelRide(rideId: String): void
+ rideStatus(rideId: String): String
interface
StripeGateway
+ charge(customerId: String, amount: Money, idempotencyKey: String): String
+ refund(chargeId: String, amount: Money): String
+ createCustomer(email: String): String
interface
FraudObserver
+ onFlagged(report: FraudReport): void
interface
TripObserver
+ onTripChanged(trip: Trip, status: TripStatus): void
interface
PricingConfiguration.FareTerms
+ baseFare: Money
+ perKm: Money
+ perMinute: Money
+ minimumFare: Money
+ surgeCap: double
+ nightSurcharge: Money
+ FareTerms(baseFare: Money, perKm: Money, perMinute: Money, minimumFare: Money, surgeCap: double, nightSurcharge: Money)
+ FareTerms(baseFare: Money, perKm: Money, perMinute: Money, minimumFare: Money)
+ FareTerms()
+ baseFare(): Money
+ perKm(): Money
+ perMinute(): Money
+ minimumFare(): Money
+ surgeCap(): double
+ nightSurcharge(): Money
+ equals(Object): boolean
+ hashCode(): int
+ toString(): String
interface
PricingService.FareBreakdown
+ baseFare: Money
+ distanceCharge: Money
+ timeCharge: Money
+ surgeMultiplier: double
+ nightSurcharge: Money
+ total: Money
+ FareBreakdown(baseFare: Money, distanceCharge: Money, timeCharge: Money, surgeMultiplier: double, nightSurcharge: Money, total: Money)
+ FareBreakdown(baseFare: Money, distanceCharge: Money, timeCharge: Money, surgeMultiplier: double, nightSurcharge: Money)
+ FareBreakdown()
+ baseFare(): Money
+ distanceCharge(): Money
+ timeCharge(): Money
+ surgeMultiplier(): double
+ nightSurcharge(): Money
+ total(): Money
interface
PricingService
+ id: UUID
+ name: String
+ city: String
+ terms: Terms
+ surgeRules: List<SurgeRule>
+ createdAt: Instant
+ updatedAt: Instant
+ PricingService(id: UUID, name: String, city: String, terms: Terms, surgeRules: List<SurgeRule>, createdAt: Instant, updatedAt: Instant)
+ PricingService(id: UUID, name: String, city: String, terms: Terms, surgeRules: List<SurgeRule>, createdAt: Instant)
+ PricingService(id: UUID, name: String, city: String, terms: Terms, surgeRules: List<SurgeRule>)
+ PricingService()
+ id(): UUID
+ name(): String
+ city(): String
+ terms(): Terms
+ surgeRules(): List<SurgeRule>
+ createdAt(): Instant
+ updatedAt(): Instant
+ equals(Object): boolean
+ hashCode(): int
+ toString(): String
interface
PaymentService
- primary: PaymentGateway
- backup: PaymentGateway
- configLoader: DeploymentConfigLoader
- clock: TripClock
- idempotencyKeys: Map<String, Integer>
- receipts: Map<UUID, Receipt>
- defaultCurrency: CurrencyCode
+ PaymentService(primary: PaymentGateway, backup: PaymentGateway, configLoader: DeploymentConfigLoader, clock: TripClock)
+ PaymentService(primary: PaymentGateway, backup: PaymentGateway, configLoader: DeploymentConfigLoader)
+ PaymentService(primary: PaymentGateway, backup: PaymentGateway)
+ authorize(rideId: String, rider: Rider, amount: Money): String
+ capture(authId: String, finalAmount: Money): Receipt
+ refund(receiptId: UUID, amount: Money): Receipt
+ receipt(id: UUID): Optional<Receipt>
+ receiptsFor(rider: Rider): List<Receipt>
+ primary(): PaymentGateway
+ backup(): PaymentGateway
+ configLoader(): DeploymentConfigLoader
+ clock(): TripClock
interface
DeploymentConfiguration
+ get(key: String): String
+ int(key: String, fallback: int): int
+ decimal(key: String): BigDecimal
+ bool(key: String): boolean
+ reload(): void
+ source(): ConfigSource
+ DeploymentConfiguration(source: ConfigSource)
+ DeploymentConfiguration()
+ loadFrom(path: Path): DeploymentConfiguration
+ loadDefault(): DeploymentConfiguration
+ loadFromClasspath(resource: String): DeploymentConfiguration
+ fromMap(values: Map<String, String>): DeploymentConfiguration
+ fromEnvironment(prefix: String): DeploymentConfiguration
+ overlay(other: DeploymentConfiguration): DeploymentConfiguration
+ withOverrides(overrides: Map<String, String>): DeploymentConfiguration
+ withOverride(key: String, value: String): DeploymentConfiguration
+ withoutKey(key: String): DeploymentConfiguration
+ keys(): Set<String>
+ asMap(): Map<String, String>
+ size(): int
+ isEmpty(): boolean
+ equals(Object): boolean
+ hashCode(): int
+ toString(): String
FraudReport
+ id: UUID
+ tripId: UUID
+ riderId: RiderId
+ amount: Money
+ score: FraudScore
+ decision: Decision
+ reasons: List<String>
+ reviewedBy: String
+ reviewedAt: Instant
+ notes: String
+ FraudReport(id: UUID, tripId: UUID, riderId: RiderId, amount: Money, score: FraudScore, decision: Decision, reasons: List<String>, reviewedBy: String, reviewedAt: Instant, notes: String)
+ FraudReport(id: UUID, tripId: UUID, riderId: RiderId, amount: Money, score: FraudScore, decision: Decision, reasons: List<String>)
+ FraudReport()
+ id(): UUID
+ tripId(): UUID
+ riderId(): RiderId
+ amount(): Money
+ score(): FraudScore
+ decision(): Decision
+ reasons(): List<String>
+ reviewedBy(): String
+ reviewedAt(): Instant
+ notes(): String
+ equals(Object): boolean
+ hashCode(): int
+ toString(): String
interface
Address
+ id: UUID
+ label: String
+ line1: String
+ line2: String
+ city: String
+ region: String
+ postalCode: String
+ countryCode: String
+ Address(id: UUID, label: String, line1: String, line2: String, city: String, region: String, postalCode: String, countryCode: String)
+ Address(id: UUID, label: String, line1: String, line2: String, city: String, region: String, postalCode: String)
+ Address()
+ id(): UUID
+ label(): String
+ line1(): String
+ line2(): String
+ city(): String
+ region(): String
+ postalCode(): String
+ countryCode(): String
+ equals(Object): boolean
+ hashCode(): int
+ toString(): String

GoCycle — Configuration

Detailed implementation view. Underlined members are static; record accessors are public.

